Relatório de benchmarking interpretado — Rota Vital

História: PI3-113 (W14 — RSD, Unidade 2) · **Depende de:** PI3-31 (coleta) e PI3-112 (telemetria simulada) **Responsável:** Lucas Ferreira Torres de Albuquerque · **Data:** 06/11/2026

> **Projeto acadêmico.** Todos os dados de domínio são sintéticos (LGPD). A telemetria de temperatura e GPS é **simulada**: não existe sensor, veículo nem hardware real (ver seção 7). As medições vêm de uma VM compartilhada de pequeno porte e não representam um ambiente hospitalar real. A compatibilidade ABO/Rh e a avaliação de cadeia fria são didáticas e não substituem protocolo clínico.

---

1. Resumo

Seis métricas, medidas na etapa de **10 req/s**, em 3 repetições válidas (média ± desvio-padrão entre repetições).

Métrica	Resultado	O que significa
Latência	p50 **204,8 ± 0,5 ms** · p95 **408,1 ± 0,8 ms** · p99 **411,0 ± 3,0 ms**	~98% do tempo é a distância Recife → Oregon (RTT de 201,6 ms). A aplicação gasta ~3 ms. O p95 alto vem de um endpoint específico (`/bolsas`), não de instabilidade.
Vazão	**10,02 ± 0,01 req/s** (66,4 KB/s); 0–1 iteração descartada	A aplicação acompanhou a carga pedida sem filas. O teto de capacidade **não foi atingido** e, portanto, não é conhecido.
Perda	Aplicação: **0%**. Rede (sonda TCP): **0%** em repouso, **1,8 ± 2,3%** sob carga	Nenhuma requisição foi perdida. A perda de rede sob carga existe, mas o TCP a converte em atraso. O valor oscila muito entre repetições e é inconclusivo.
Jitter	Rede: **0,64 ± 0,09 ms** · Aplicação: **0,82 ± 0,30 ms**	Comunicação muito estável. É irrelevante perto do intervalo de 30 s entre leituras de telemetria.
Erros 4xx	**0%**	Nenhuma requisição mal formada ou negada no caminho de leitura medido.
Erros 5xx	**0%**	Nenhuma falha do servidor nas rodadas válidas. O modo de falha conhecido (504 por cold start ou pool esgotado) ficou fora do perfil de carga, de propósito.

Conclusão em uma frase: no ambiente atual, quem decide o tempo de resposta do Rota Vital é a **rede** (a distância até a VM), não a aplicação. Com carga de leitura de até 10 req/s, o sistema respondeu de forma estável, sem erros e sem perda.

---

2. Ambiente medido

Item	Valor
Aplicação	API Spring Boot do Rota Vital atrás do Nginx (TLS/Certbot), banco **H2 em memória**
Servidor	VM única no GCP (`flux` , zona `us-west1-a` , Oregon, EUA), **978 MB de RAM**, compartilhada com outra aplicação (Flux)
JVM	`-Xmx256m` , coletor **SerialGC** (pausa a aplicação inteira a cada coleta)
Endereço medido	`https://rsc3-rotavital.duckdns.org` (porta 443)
Origem da carga	Notebook da equipe em Recife-PE, rede residencial, Windows 11
Ferramentas	k6 v2.2.0 · Python 3.12.10 (`sonda_tcp.py` , `analisar_rede.py`)
Data	25/10/2026, entre 20h29 e 21h37 (horário de Brasília)

A maior parte dos resultados se explica pela distância entre a origem (Recife) e a VM (Oregon), de ~11 mil km em linha reta. Só a ida e volta de um pacote leva ~201 ms.

---

3. Método de coleta

O método completo, com scripts e justificativas, está em `docs/benchmark-rede.md` (PI3-31). Aqui fica o necessário para entender e repetir a medição.

3.1 Perfil de carga

```
|-- aquecimento --|---- etapa 1 ----|---- etapa 2 ----|-- repouso --|
     1 req/s         5 req/s           10 req/s         só sondas
   (descartado)      (medido)          (medido)
      120 s            300 s             300 s            60 s
```

- **Taxa de chegada fixa** (executor `constant-arrival-rate` do k6): as requisições saem no ritmo combinado, mesmo que o servidor engasgue. Isso evita a *coordinated omission*, o viés que faz o p99 parecer melhor do que é.
- **Endpoints, só leitura:** `GET /api/v1/hemocentros` (30%), `GET /api/v1/bolsas?status=DISPONIVEL` (30%), `GET /api/v1/indicadores/estoque-por-tipo` (20%) e `GET /api/v1/indicadores/validade` (20%). Escrever no H2 mudaria o estado entre repetições, e elas deixariam de ser comparáveis.
- **Em paralelo durante toda a rodada:** `GET /actuator/health` a 2 req/s (base do jitter de aplicação), um **canário** que faz `GET` na outra aplicação da VM a cada 10 s, e a **sonda TCP**, rodando 60 s em repouso antes da carga e de novo durante as etapas.
- **Proteções:** no máximo 40 usuários virtuais. A rodada é interrompida automaticamente se o p95 passar de 3 s, se os erros passarem de 5% ou se o p95 da outra aplicação passar de 3 s.

3.2 Como cada métrica foi coletada

Métrica	Unidade	Ferramenta	Definição
Latência	ms	k6 (`http_req_duration`)	Do envio da requisição ao último byte da resposta, visto pelo cliente. Reportada em p50/p95/p99/máximo
Vazão	req/s, KB/s	k6 (`http_reqs`, `data_received`)	Requisições concluídas na etapa ÷ duração observada da etapa
Perda (aplicação)	%	k6	Requisições sem nenhuma resposta HTTP (timeout, recusa ou reset)
Perda (rede)	%	`sonda_tcp.py`	Handshakes TCP na porta 443 sem resposta em 1 s
Jitter (rede)	ms	`sonda_tcp.py`	Média de $\lvert RTT(i) - RTT(i-1) \rvert$ entre amostras consecutivas (RFC 3550)
Jitter (aplicação)	ms	k6, cenário `health`	Mesma fórmula, aplicada à latência do `/actuator/health`
Erros 4xx / 5xx	%	k6	Respostas 4xx (ou 5xx) ÷ total de respostas

Por que uma sonda TCP e não `ping`: quem responde ao SYN é o kernel da VM, e não o Nginx nem a JVM. Assim a sonda isola a rede, usando o mesmo caminho e a mesma porta (443) do tráfego real. Os runners do GitHub Actions também bloqueiam ICMP.

3.3 Repetições

Rodada	Início (Brasília)	Situação
rep1	25/10 20h29	**Excluída**: interrompida pela proteção de p95 > 3 s após 81 s da etapa 1 (ver 4.1)
rep2	25/10 20h36	Válida
rep3	25/10 20h56	Válida
rep4	25/10 21h22	Válida

Os dados brutos de cada rodada (incluindo a excluída) estão em `benchmark/dados-brutos/`. O consolidado está em `benchmark/resumo-rede.json` e `.csv`.

3.4 Como repetir

```
ORIGEM=notebook-recife REPETICAO=1 bash benchmark/coletar.sh   # ~13 min por rodada
python3 benchmark/analisar_rede.py --publicar                  # gera o resumo e publica no painel
```

Antes de medir: congelar deploys (o deploy reinicia a API e zera o H2), deixar o script aquecer a JVM (o cold start leva 20–30 s e pode devolver 504) e espaçar as rodadas em 20–30 min. A rodada não exige acesso à VM.

---

4. Resultados e interpretação

4.1 Latência

Indicador (3 repetições válidas)	Etapa 1 (5 req/s)	Etapa 2 (10 req/s)
p50	205,0 ± 0,7 ms	204,8 ± 0,5 ms
p95	408,6 ± 0,9 ms	408,1 ± 0,8 ms
p99	411,7 ± 2,6 ms	411,0 ± 3,0 ms
RTT da rede (sonda, p50)	201,6 ms	201,6 ms

Para onde vai o tempo. O p50 da aplicação (204,8 ms) menos o RTT da rede (201,6 ms) dá **~3 ms**. Essa é a soma de todo o trabalho do Nginx, do Spring Boot e do H2 numa requisição típica. Os outros ~98% são a viagem do pacote até Oregon e de volta. Melhorar código ou consultas mal mexeria nesse número. O que mexeria seria hospedar a aplicação mais perto dos usuários, por exemplo numa região do Brasil (`southamerica-east1`).

Dobrar a carga não mudou a latência. De 5 para 10 req/s, o p50 e o p95 ficaram iguais dentro da variação entre repetições. A aplicação está longe da saturação nessa faixa.

O p95 de ~408 ms tem uma causa específica. Ele não indica instabilidade. Vem de um endpoint:

Endpoint (etapa 2, rep2–rep4)	p50
`GET /api/v1/hemocentros`	~204 ms (1 RTT)
`GET /api/v1/indicadores/estoque-por-tipo`	~204 ms (1 RTT)
`GET /api/v1/indicadores/validade`	~204 ms (1 RTT)
`GET /api/v1/bolsas?status=DISPONIVEL`	**~407 ms (2 RTT)**

A listagem de bolsas responde em exatamente **duas idas e voltas**. Como ela representa 30% da carga, o p90/p95 geral cai sobre ela. Um custo de servidor dificilmente somaria exatamente mais um RTT, então a hipótese mais provável é de rede: essa listagem **não é paginada** (`BolsaController.listar` devolve a lista inteira) e a aplicação não tem compressão configurada. A resposta, maior que as outras, não cabe na primeira janela de transmissão do TCP e precisa de mais uma ida e volta. **Não foi verificado** com captura de pacotes. Para confirmar, compare o tamanho da resposta com o dos outros endpoints, ou teste com paginação/gzip e veja se o endpoint cai para 1 RTT.

O que isso significa para a entrega de hemocomponentes. Para quem está num hemocentro ou hospital, o efeito aparece na tela de **estoque disponível**, a consulta feita antes de alocar uma bolsa. Ela custa o dobro das outras (~0,4 s contra ~0,2 s). Nas consultas de rotina, a diferença é imperceptível. Numa requisição de **emergência**, em que o operador navega por várias telas seguidas, os atrasos se somam: cinco chamadas em sequência ficam entre 1 e 2 s, e o grosso disso é distância. Mesmo assim, é um tempo pequeno perto do tempo físico de transporte de uma bolsa, que é de horas. Para o fluxo atual, a latência **não é gargalo**. Ela passaria a ser se o fluxo de alocação exigisse dezenas de chamadas encadeadas.

A rodada excluída (rep1) também é informativa. Nos primeiros 81 s da etapa 1, o p95 chegou a 6 s e o máximo a 13 s. O próprio `/actuator/health`, que não faz trabalho nenhum, chegou a 12 s no p99. Ao mesmo tempo, a sonda TCP via a rede normal (p99 de 207 ms, 0% de perda). Ou seja: **quem travou foi a VM ou a JVM, não a rede.** As causas compatíveis são as da seção "Por que percentis" do método: pausas do SerialGC, falta de créditos de CPU numa VM pequena e compartilhada, ou resto do cold start. A causa exata não foi isolada (sem `vmstat` nessa rodada). A proteção de p95 > 3 s interrompeu a rodada como previsto. Para o domínio, o achado relevante é que **o maior risco de latência é a VM única**, e não a rede (ver o plano de implantação, PI3-114).

4.2 Vazão

Indicador	Etapa 1	Etapa 2
Vazão alcançada	5,01 req/s	10,02 ± 0,01 req/s
Volume recebido	~33 KB/s	66,4 ± 0,5 KB/s (~6,6 KB por requisição)
Iterações descartadas pelo k6	0	0–1

Leitura. A vazão entregue foi igual à pedida, e o k6 quase não precisou descartar iterações por falta de usuário virtual livre (o sinal de saturação). A aplicação **dá conta de 10 req/s de leitura** com folga. Esse número **não é a capacidade máxima**: o teste foi limitado de propósito a 10 req/s para não prejudicar a outra aplicação da VM, e o teto real não foi procurado.

Para hemocomponentes. 10 req/s equivalem a 36 mil consultas por hora. Para comparação, o simulador de telemetria gera ~0,1 req/s, e a operação de uma rede regional de hemocentros e hospitais com usuários humanos gera poucas requisições por minuto em cada unidade. A vazão medida cobre a operação didática atual com margem grande. O que limita a vazão nesta arquitetura não é a rede (66 KB/s é pouco), e sim a VM com 256 MB de heap e pool de 10 conexões. A operação de escrita em lote (`POST /api/v1/bolsas/lote`, PI3-30) esgota esse pool com três lotes simultâneos. Ela ficou fora do teste e seria o primeiro ponto a medir num teste de capacidade.

4.3 Perda

Indicador	Repouso	Sob carga (rep2 / rep3 / rep4)
Perda na aplicação (sem resposta HTTP)	—	**0% / 0% / 0%**
Perda na rede (sonda TCP)	0% / 0% / 0%	**4,48% / 0,41% / 0,62%** (média 1,84 ± 2,29%)

Leitura. São dois níveis diferentes:

- **Aplicação: nenhuma requisição ficou sem resposta** em 9.000 requisições medidas na etapa 2.
- **Rede: alguns handshakes TCP não foram respondidos em 1 s**, mas só durante a carga, nunca em repouso. O TCP retransmite pacotes perdidos, então essa perda não vira erro, vira **atraso**. Isso explica a perda de rede com perda de aplicação zero.

O valor de rede é **inconclusivo**: varia 10 vezes entre repetições (4,48% contra 0,41%). Uma explicação possível é a sonda rodar no **mesmo**

notebook que gera a carga, numa rede residencial. Parte da perda pode estar no próprio Wi-Fi ou no notebook ocupado, e não no caminho até a VM. Rodadas a partir do GitHub Actions, previstas no plano de coleta, separariam as duas causas.

Para hemocomponentes. A perda importa sobretudo para a **telemetria de cadeia fria**. Um sensor que abre uma conexão nova a cada leitura pagaria uma retransmissão (~1 s a mais) em cerca de 0,4% a 4,5% das leituras sob carga. A leitura chega atrasada, não se perde. Com leituras a cada 30 s, 1 s de atraso não muda nenhuma decisão. Seria diferente com um protocolo sem retransmissão (UDP/MQTT QoS 0), em que a mesma perda viraria leitura de temperatura faltando. Um alerta de "porta aberta" poderia passar sem registro. É um bom argumento para o canal atual (HTTPS/TCP), e um ponto de atenção se a telemetria migrar para outro protocolo.

4.4 Jitter

Indicador	Repouso	Etapa 2 (rep2 / rep3 / rep4)
Jitter de rede (sonda TCP)	0,53 ± 0,13 ms	0,70 / 0,68 / 0,54 ms (0,64 ± 0,09)
Jitter de aplicação (`/actuator/health`)	—	1,16 / 0,70 / 0,59 ms (0,82 ± 0,30)

Leitura. O jitter é baixo nos dois níveis: menos de 1% da latência. A carga quase não o alterou (de 0,53 para 0,64 ms na rede). A diferença entre o jitter de aplicação e o de rede (~0,2 ms) é a variação introduzida pelo Nginx e pela JVM, que também é pequena. A exceção foi a rep1, com jitter de aplicação de **162 ms**, o mesmo travamento da VM descrito em 4.1. **Jitter alto neste sistema é sintoma da VM, não da rede.**

Para hemocomponentes. O jitter afeta a **regularidade** da telemetria: leituras que deveriam chegar a cada 30 s chegariam a intervalos irregulares, e uma regra de alerta baseada em "N leituras seguidas fora da faixa" ficaria imprecisa. Com jitter abaixo de 1 ms contra um intervalo de 30.000 ms, o efeito é nulo. Ele só seria relevante com leituras muito mais frequentes (frações de segundo) ou com voz/vídeo, que não fazem parte do sistema.

4.5 Erros 4xx

Resultado: 0% nas três repetições válidas (e também na rep1). Nenhum 409 (conflito).

Leitura. Todas as requisições do perfil eram válidas e não exigiam autenticação, então 0% era o esperado. O número confirma que o **caminho de leitura está correto**: rotas, parâmetros e serialização funcionam sob carga. Ele **não** diz nada sobre o caminho de escrita. O 409 de conflito na alocação de bolsas (PI3-108) foi excluído de propósito: ele é o comportamento correto quando duas requisições disputam a mesma bolsa e inflaria a taxa sem indicar falha.

Para hemocomponentes. Num teste com escrita, um 4xx teria significados diferentes conforme o código. Um 409 seria **bom sinal**: o sistema impediu que a mesma bolsa fosse alocada duas vezes. Um 400 ou 404 seria erro do cliente, por exemplo um hospital enviando um tipo de hemocomponente inválido. Por isso o 409 é contado e mostrado à parte.

4.6 Erros 5xx

Resultado: 0% nas três repetições válidas (e também na rep1).

Leitura. Nenhuma falha do lado do servidor. Mesmo a rep1, com latências de 13 s, não chegou a gerar 504, porque o Nginx só desiste após 30 s. Os modos de falha 5xx **conhecidos** ficaram fora do perfil, de propósito: cold start (tratado pelo aquecimento) e esgotamento do pool de conexões pela escrita em lote. Então 0% significa "nenhum 5xx com leitura a até 10 req/s, aplicação aquecida", e não "a aplicação nunca devolve 5xx".

Para hemocomponentes. O 5xx é o erro mais grave para o domínio. Ele significa que um hospital pediu algo e o sistema não conseguiu responder, sem culpa do hospital. Numa requisição de emergência, um 504 obrigaria o operador a tentar de novo sem saber se a primeira tentativa chegou a reservar a bolsa. O risco real hoje é o **reinício da VM ou da aplicação**: cada deploy reinicia a API, zera o H2 (todas as bolsas e requisições somem) e expõe o cold start. Esse risco é tratado no plano de implantação (PI3-114), não neste teste.

---

5. Síntese: rede versus aplicação

Camada	Quanto pesa	Evidência
Rede (distância até Oregon)	~201 ms por ida e volta, ~98% da latência típica	RTT da sonda estável em 201,6 ms com e sem carga
Tamanho de resposta (hipótese)	+1 RTT (~203 ms) em `/bolsas`	Endpoint não paginado responde em exatamente 2 RTT
Aplicação (Nginx + Spring Boot + H2)	~3 ms em funcionamento normal	p50 HTTP − RTT da sonda
VM única e compartilhada	Até 13 s em episódios de travamento	rep1: `health` lento com rede normal

As três ações com mais efeito, em ordem: **(1)** reduzir o risco da VM única (o pior caso observado); **(2)** paginar ou comprimir a listagem de bolsas (metade do p95); **(3)** hospedar mais perto dos usuários. Otimizar o código da API, sozinho, mexeria em ~3 ms.

---

6. Impacto na outra aplicação da VM

Critério de aceite da PI3-31: a medição não pode prejudicar as outras aplicações da VM.

Canário (outra aplicação, p95)	Antes	Durante	Depois
Média das 3 repetições válidas	617 ms	682 ms	665 ms
rep2 / rep3 / rep4	630 / 611 / 610	627 / **806** / 612	**773** / 611 / 610

A mediana da outra aplicação ficou estável (~609 ms, ou 3 RTT, em todos os momentos). Houve picos isolados de ~800 ms no p95 de duas repetições, um durante e um depois da carga, e nenhum perto do limite de 3 s que interromperia a rodada. O impacto foi **pequeno e pontual**.

---

7. Declaração sobre a telemetria simulada

A telemetria de temperatura e GPS do Rota Vital é simulada. Não há sensor, veículo nem hardware real. O programa `telemetria/simulador.py` (PI3-112) faz o papel dos sensores de 3 veículos e envia uma leitura sintética de temperatura e posição a cada 30 s para `POST /api/v1/rotas/{id}/temperaturas`, pela internet, passando por Nginx e API como um sensor real passaria. A temperatura segue um modelo que volta ao centro da faixa do componente, com 2% de chance de "porta aberta" por leitura para gerar alertas. O GPS é uma interpolação em linha reta entre origem e destino, com ruído de ~15 m. Detalhes em `docs/telemetria-simulada.md`.

O que isso significa para estes números:

- As medições deste relatório (25/10) usaram **carga sintética de leitura gerada pelo k6**, com o cenário de telemetria **desligado** (`"telemetria": false` em todos os `meta.json`). O canal de telemetria simulada da PI3-112 foi entregue depois, em 28/10.
- Portanto, o **endpoint de telemetria não foi medido sob carga**. As interpretações sobre telemetria nas seções 4.3 e 4.4 são extrapolações a partir da rede medida, não medições do canal.
- Nenhum número deste relatório representa sensores, veículos ou hospitais reais.

---

8. Limitações e pendências

Limitação	Efeito	O que fazer
Uma origem só (notebook em Recife). As 3 rodadas previstas a partir do GitHub Actions não foram feitas	Não dá para separar a perda do Wi-Fi/notebook da perda do caminho até a VM	Rodar 3 repetições pelo workflow "Benchmark de rede" (não exige acesso à VM)
Telemetria não medida (cenário desligado; canal entregue depois da coleta)	Sem número próprio para o endpoint de telemetria	Uma rodada com `TELEMETRIA=1`, agora que o canal da PI3-112 está no ar
Rodadas mais próximas que o protocolo (3 a 12 min de intervalo, contra os 20–30 min previstos)	Possível efeito de créditos de CPU entre rodadas, que pode ter afetado a rep1	Respeitar o espaçamento nas próximas rodadas
Só leitura, até 10 req/s	Teto de capacidade e caminho de escrita não conhecidos	Teste de capacidade separado, fora do horário da outra aplicação
H2 em memória	O banco medido não é o de produção (PostgreSQL entra na Entrega 02)	Repetir após a migração
Causa do travamento da rep1 não isolada	Hipóteses (GC, CPU, cold start) não confirmadas	Coletar `vmstat` na VM durante as rodadas (exige quem tem acesso)
Causa do 2º RTT em `/bolsas` não verificada	Hipótese de tamanho de resposta	Medir o tamanho da resposta ou testar com paginação/gzip

---

9. Conclusão

Com carga de leitura de até 10 req/s, o Rota Vital implantado respondeu de forma **estável e previsível**: latência típica de ~205 ms, p95 de ~408 ms, nenhum erro 4xx ou 5xx, nenhuma requisição perdida e jitter abaixo de 1 ms. A análise mostra que esse tempo é quase todo **distância de rede**, e que a parte de aplicação é pequena (~3 ms). Os dois pontos que mais afetam a entrega de hemocomponentes não estão no código da API: a listagem de estoque não paginada, que dobra o tempo da consulta feita antes de alocar uma bolsa, e a **VM única e compartilhada**, que produziu o único episódio grave observado (travamentos de até 13 s) e é o principal risco a tratar no plano de implantação (PI3-114).

Arquivos de referência

Arquivo	Conteúdo
`docs/benchmark-rede.md`	Método completo (PI3-31)
`benchmark/dados-brutos/`	Dados brutos das 4 rodadas
`benchmark/resumo-rede.json` / `.csv`	Consolidado usado neste relatório
`/painel-rede.html`	Painel de rede com os mesmos números
`docs/telemetria-simulada.md`	Canal de telemetria simulada (PI3-112)